

Project 5 Report: AI Agent Pipeline for LaTeX Project Compression

1. Objective

The objective of this project was to design and implement an AI-agent-style pipeline for optimizing a LaTeX project. The goal was to reduce the total compressed project size while preserving the visual quality of the final compiled PDF.

The project had two main requirements for undergraduate students:

- Part 1: Reduce the full LaTeX project zip size to less than 50 MB.
- Part 2: Further optimize the compiled PDF so that the PDF size is less than 30 MB.

A key constraint was that the project should not minimize size at all costs. The final result needed to balance compression with visual clarity.

2. Initial Project State

The original LaTeX source project was downloaded from the provided Overleaf link as a zip file.

Initial measurements:

Item	Size
------	------

---	---:
-----	------

Original downloaded zip	118 MB
-------------------------	--------

Original unzipped LaTeX project	119 MB
---------------------------------	--------

The original project contained several large image files, backup files, and duplicate/unused figures.

3. Pipeline Design

I designed the workflow as a multi-step agent pipeline:

Step 1: Analysis Agent

The analysis step inspected the LaTeX project folder and identified large files. It also searched the LaTeX source files for active `\includegraphics` commands to determine which images were actually used in the final PDF.

This step helped separate necessary figures from unused or backup figures.

Step 2: Decision Agent

The decision step determined which files could be safely removed or moved outside the optimized project. Files were treated as removable if they were:

- backup files,
- duplicate figures,
- unused figures,
- old figure versions,
- not referenced by active LaTeX source code.

Instead of deleting files permanently, unused figures were moved into a separate folder. This made the process safer because files could be restored if compilation failed.

Step 3: Action Agent

The action step performed the optimization. The main actions were:

- removed the `backups` folder from the optimized LaTeX project,
- removed accidental nested duplicate project folders,
- moved unused figure versions outside the final project,
- compressed remaining JPG images using moderate image compression,
- preserved images that were actively referenced in the LaTeX source.

The image compression was done using macOS `sips`, with a maximum image dimension and JPEG quality setting. The goal was to reduce image file sizes while keeping the figures visually readable.

Step 4: Verification Agent

The verification step checked whether the optimized project satisfied the requirements.

I verified:

- the optimized source zip size,
- the compiled PDF size,
- whether the LaTeX project compiled successfully,
- whether figures were still visible and visually acceptable in the final PDF.

4. Implementation Details

The pipeline was implemented in Python in `code/project5\_pipeline.py`.

The script performs the following tasks:

- computes project and file sizes,
- lists the largest files,
- scans `.tex` files for active `\includegraphics` references,
- identifies used image files,
- provides functions for moving unused files,
- provides functions for compressing JPG images,
- creates an optimized zip archive,
- sends a compact project summary to an API-based decision/verification agent,
- saves the API agent response as `report/api_agent_log.md`.

Most of the pipeline uses local deterministic steps. This includes file-size analysis, image reference detection, file movement, image compression, and zip creation. These tasks do not require a foundation model and can be handled more efficiently with Python and command-line tools.

After receiving the OpenRouter API key, I added an API-based agent step to the pipeline. The API key was not hard-coded into the Python file. Instead, it was stored as an environment variable named `OPENROUTER_API_KEY`. This made the workflow safer because the secret key was not included in the submitted code, report, or GitHub webpage.

The API agent received a compact summary of the project rather than the full LaTeX source. The summary included:

- optimized LaTeX project size,
- final compiled PDF size,
- optimized source zip size,

- active image references found in the LaTeX source,
- largest remaining files,
- actions taken during optimization,
- project requirements and quality constraints.

The agent then reviewed whether the optimization satisfied the required size constraints and whether the approach balanced compression with visual quality. The response was saved in `api\_agent\_log.md` and included in the final submission.

5. Budget-Aware AI Strategy

Because the project provides a fixed API budget, my strategy was to avoid using the API for tasks that could be solved locally. I did not send the entire LaTeX project, all images, or the full PDF to the model. Instead, the pipeline used deterministic Python steps for file analysis and compression, then used the API only for a higher-level decision and verification step.

This approach reduced unnecessary API usage. The API agent only received a compact text summary of the project, including file sizes, active figure references, largest files, and actions taken. This allowed the model to act as a decision/verification agent without consuming excessive budget.

The API-based agent was used to:

- review whether the optimized project satisfied the size requirements,
- evaluate whether the compression strategy was reasonable,
- identify possible risks or limitations,
- provide a concise recommendation for the final report.

This budget-aware design reflects the main goal of the project: building an agentic workflow that combines deterministic tools with foundation models, rather than relying on the model for every step.

6. Final Results

After optimization, the final project satisfied both requirements.

Item	Size / Status
---	---
Original downloaded zip	118 MB
Original unzipped LaTeX project	119 MB
Optimized LaTeX source zip	16 MB
Final compiled PDF	17 MB
Required final zip size	< 50 MB
Required PDF size	< 30 MB
API agent log	Included as `api_agent_log.md`

The optimized source zip was reduced from 118 MB to 16 MB. The final compiled PDF was 17 MB, which is below the 30 MB requirement. The API-based agent log was also included to document the decision/verification stage of the pipeline.

7. Visual Quality Check

After compiling the optimized LaTeX project, I opened and inspected the final PDF. I checked that:

- figures were not missing,
- image comparison tables remained visible,

- the main illustration figures were still readable,
- the layout was not corrupted,
- there were no severe compression artifacts.

Because the project emphasizes balancing compression and quality, I stopped further compression once the size requirements were safely met.

8. Conclusion

This project demonstrated how an agentic AI workflow can be applied to a practical document optimization task. The final pipeline followed an analysis, decision, action, and verification structure.

The project also showed that not every part of an AI-agent pipeline needs to use an expensive model. Local tools can handle file analysis, image reference detection, compression, and zip creation. The API-based agent was reserved for the judgment-based part of the workflow: reviewing the optimization strategy, checking whether the size requirements were satisfied, and identifying possible risks.

The final optimized LaTeX project and compiled PDF satisfied the required size limits while preserving acceptable visual quality. The API key was used safely through an environment variable and was not included in the submitted code or report.